The Hop-Count Cliff: Characterizing the Breakdown of Graph Retrieval Strategies on Dense Knowledge Graphs

Matt Krasnow Seager Hunt Cory Wu Reade Park
Stat 175: Networks
Harvard University

Spring 2026

Abstract

We investigate how retrieval strategy affects answer quality in graph retrieval-augmented generation (GraphRAG) as query complexity increases. Through controlled experiments on two STaRK benchmarks—PrimeKG (129K nodes, mean degree 62.6, 11,204 queries) and Amazon (1M nodes, mean degree 9.1, 9,100 queries)—we compare six retrieval strategies stratified by query hop count. On the dense PrimeKG graph, we find a sharp *performance cliff*: Hit@10 drops ∼70% from 1-hop to 2-hop queries across all strategies. On the sparse Amazon graph, the degradation is gradual (∼40% drop), and strategies that exploit graph structure *underperform* text-only retrieval at multi-hop—the opposite of dense-graph behavior. This cross-domain comparison reveals that **cliff severity is governed by graph density**: the combinatorial explosion of the neighborhood at each hop overwhelms embedding-based retrieval on dense graphs but is manageable on sparse ones. Ablation studies reveal a fundamental focus-versus-coverage tradeoff, and two principled approaches to overcoming the cliff—beam search and query-adaptive routing—fail, confirming the barrier is intrinsic to embedding-based retrieval on dense graphs.

1 Introduction

Retrieval-augmented generation (RAG) has become a standard approach for grounding large language model (LLM) outputs in external knowledge [Lewis et al., 2020]. While traditional RAG retrieves text passages via vector similarity, *graph* RAG (GraphRAG) leverages the relational structure of knowledge graphs (KGs) to retrieve more contextually relevant information. Recent work has proposed diverse retrieval strategies including subgraph extraction [Hu et al., 2024], path-based reasoning [Yu et al., 2025], and hybrid structural-textual mixtures [Yue et al., 2025].

A natural question arises: *when does graph structure actually help?* All three approaches report improvements over text-only baselines, but none systematically varies query complexity to identify the boundary where structural retrieval becomes necessary. Yue et al. [2025] report "uneven retrieving performance across different query logics" without characterizing the unevenness, and Yu et al. [2025] critique retrieval fidelity without isolating hop count as a variable.

We address this gap with a controlled experiment that isolates query hop count—the number of relational hops from the query entity to the answer—as the independent variable. We hypothesized that a phase transition exists at some critical hop count h^* where structural retrieval overtakes text-only retrieval. Our findings are more striking: rather than a gradual crossover, we observe a *cliff*—a catastrophic drop in performance from 1-hop to 2-hop queries for *all* strategies, followed by a low plateau. The phase transition exists not between strategies but between solvable and nearly unsolvable queries.

Our contributions are:

1. We characterize the **hop-count cliff**: a sharp performance boundary at 2 hops on dense text-rich KGs where all retrieval strategies collapse.
2. We provide statistical evidence (paired bootstrap tests, logistic regression interaction analysis) that structural methods **gain relative advantage** as query complexity increases, even though absolute performance is low.
3. Through five ablation studies, we identify a **fundamental focus-vs-coverage tradeoff** that no single hyperparameter setting resolves.
4. We show that two principled approaches to overcoming the cliff—**beam search** and **query-adaptive routing**—fail, demonstrating that the barrier is intrinsic to embedding-based retrieval on dense graphs.
5. We replicate on STaRK-Amazon (sparse, $\bar{d} = 9.1$), showing that **cliff severity correlates with graph density** and that structural retrieval can *hurt* on sparse graphs—establishing density as the key moderating variable.

2 Related Work

Graph RAG. Hu et al. [2024] introduce GRAG, which retrieves subgraphs around query-relevant entities and encodes them via a graph neural network before passing to an LLM. They demonstrate improvements on commonsense reasoning (ExplaGraphs) and multi-hop QA (WebQSP) but do not analyze performance as a function of hop count.

Hybrid Retrieval. Yue et al. [2025] propose MoR (Mixture of Retrieval), which interweaves structural traversal and textual matching through a Planning-Reasoning-Organizing framework. They evaluate on STaRK [Wu et al., 2024] and note uneven performance across query logics, motivating our systematic investigation of this unevenness.

Retrieval Fidelity. Yu et al. [2025] critically examine whether KG-based RAG actually retrieves what is needed, introducing GraphFlow, a flow-based retrieval method. They show existing methods often fail to retrieve relevant context but focus on proposing a solution rather than characterizing when and why failure occurs.

Iterative and Adaptive Retrieval. Sun et al. [2024] propose Think-on-Graph, which uses an LLM to guide beam search over a KG, selecting which edges to follow at each hop. Wang et al. [2025] introduce SkewRoute, a training-free method that uses the skewness of retrieval score distributions to route queries to different processing pipelines. Trivedi et al. [2023] interleave retrieval with chain-of-thought reasoning, retrieving new context at each reasoning step. These approaches motivate our beam search and adaptive routing experiments.

STaRK Benchmark. Wu et al. [2024] introduce STaRK, a benchmark for semi-structured retrieval over text-rich knowledge bases covering three domains (biomedical, e-commerce, academic). We use STaRK-PrimeKG as our primary evaluation setting, as it is used by both MoR and GraphFlow, enabling direct comparison.

3 Experimental Design

3.1 Datasets

We use two STaRK benchmarks [Wu et al., 2024] to compare across graph densities:

- **STaRK-PrimeKG** (biomedical): Derived from PrimeKG [Chandak et al., 2023], containing drugs, diseases, genes, and pathways. Dense graph with high-degree hub nodes characteristic of biological networks. 11,204 QA pairs.
- **STaRK-Amazon** (e-commerce): Product nodes connected by co-purchase, brand, category, and color relationships. Sparse graph with many small connected components. 9,100 QA pairs.

Table 1: Graph properties of the two STaRK benchmarks.

Property	PrimeKG	Amazon
Nodes	129,375	1,035,542
Edges	4,049,642	4,722,008
Mean degree	62.6	9.1
Median degree	4.0	3.0
Max degree	17,355	64,062
QA pairs	11,204	9,100

The key contrast is density: PrimeKG's mean degree (62.6) is $\sim$7$\times$ higher than Amazon's (9.1). This means a 2-hop expansion on PrimeKG touches $\sim$3,600 nodes on average, while on Amazon it touches only $\sim$80—a 45$\times$ difference in neighborhood size that we hypothesize drives the cliff severity.

3.2 Hop-Count Stratification

We stratify the 11,204 QA pairs by the shortest path length from the query entity to the gold answer in the knowledge graph. Since STaRK does not provide explicit query entity IDs, we identify the query entity as the graph node whose text embedding is most similar to the query (using all-MiniLM-L6-v2 [Reimers and Gurevych, 2019]). We then compute the shortest path to each gold answer node via BFS.

Table 2: Distribution of QA pairs by hop count.

Hop Count	n	% of Total
1-hop	3,851	34.4%
2-hop	3,699	33.0%
3-hop	2,193	19.6%
4-hop	1,001	8.9%
5-hop	313	2.8%
6+-hop	147	1.3%

All bins contain $\geq$ 50 samples, providing sufficient statistical power for pairwise tests.

3.3 Retrieval Strategies

We implement four retrieval strategies, each taking a natural language query and returning a ranked list of candidate node IDs:

1. **Node-centric** (text-only baseline): Retrieve the top-k nodes by cosine similarity between the query embedding and node text embeddings. This ignores graph structure entirely.
2. **Path-centric** (inspired by GraphFlow [Yu et al., 2025]): Identify seed nodes via embedding similarity, compute shortest paths between all seed pairs, score paths by average node-query similarity, and return nodes from the highest-scoring paths.
3. **Subgraph-centric** (inspired by GRAG [Hu et al., 2024]): Identify seed nodes, expand by k hops, cap the subgraph size via relevance-based pruning, and return nodes ranked by embedding similarity within the subgraph.
4. **Hybrid** (inspired by MoR [Yue et al., 2025]): Combine textual similarity scores and structural proximity scores with a tunable weight parameter $\alpha \in [0, 1]$.
5. **Beam search** (inspired by Think-on-Graph [Sun et al., 2024]): Iterative hop-by-hop expansion with embedding-guided pruning. At each hop, expand all neighbors of the current beam, score by embedding similarity, and retain only the top-b (beam width) nodes. This converts exponential expansion (d^k for degree d and k hops) to linear ($b \times k$).
6. **Adaptive router** (inspired by SkewRoute [Wang et al., 2025]): Compute the skewness of the top-50 embedding similarity scores for each query. High skewness indicates a focused query (one dominant seed node), routing to a focused configuration (1 seed, 1-hop expansion). Low skewness indicates a diffuse query, routing to a broad configuration (5 seeds, 2-hop expansion). This tests whether query-adaptive strategy selection can overcome the cliff.

3.4 Controls

All strategies use the same embedding model (all-MiniLM-L6-v2) and shared FAISS index. We evaluate retrieval quality directly via Hit@k and MRR rather than end-to-end generation quality, isolating the retrieval component.

3.5 Statistical Methods

- **Paired bootstrap tests** ($B = 10{,}000$) for pairwise strategy comparisons at each hop count.
- **Logistic regression** with features (hop count, is_structural, hop $\times$ is_structural) to test the interaction between strategy type and query complexity on Hit@1.
- **Cohen's** d effect sizes to quantify practical significance.

4 Results

4.1 The Hop-Count Cliff

Figure 1 shows retrieval performance as a function of hop count for all four strategies. The dominant feature is a *cliff*: Hit@10 drops approximately 70% from 1-hop to 2-hop across all strategies, then plateaus at low values through 6+ hops.

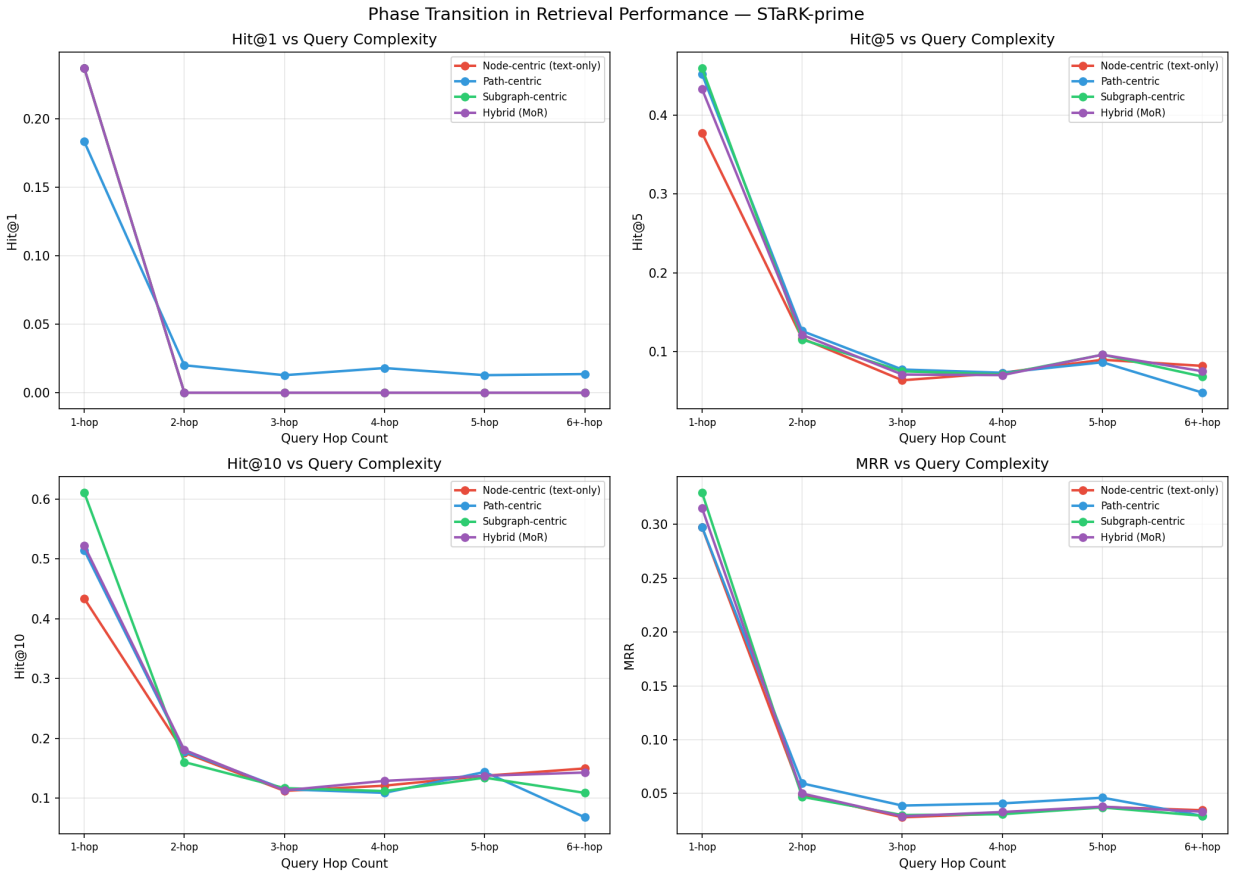

Figure 1: Retrieval performance vs. query hop count for all four strategies on STaRK-PrimeKG ($n = 11{,}204$). A sharp cliff appears between 1-hop and 2-hop queries.

Table 3 presents the full results. At 1-hop, subgraph-centric retrieval achieves the highest Hit@10 (0.611), substantially outperforming the node-centric baseline (0.434). At 2+ hops, all strategies perform poorly, though path-centric is the only strategy with Hit@1 > 0.

4.2 Statistical Analysis

Interaction effect. Logistic regression on Hit@1 with features (hop count, is_structural, hop $\times$ structural) yields an interaction coefficient of $\beta = +2.09$, confirming that structural methods gain relative advantage as hop count increases.

Pairwise tests. Paired bootstrap tests ($B = 10{,}000$) show that path-centric retrieval significantly outperforms node-centric on Hit@1 at 2-hop through 5-hop ($p < 0.05$, Cohen's $d = 0.16$–0.20). However, the absolute differences are small (1–2 percentage points). Subgraph-centric retrieval significantly outperforms node-centric on MRR at 1-hop ($p < 0.001$, $d = 0.08$).

4.3 Ablation Studies

We conduct five ablation studies on a subsample of 200 queries per hop-count bin to understand the sensitivity of each strategy to its key hyperparameters.

Table 3: Hit@10 and MRR by hop count and retrieval strategy. Bold indicates best per row.

	Hit@10				MRR			
Hop	Node	Path	Subgraph	Hybrid	Node	Path	Subgraph	Hybrid
1	0.434	0.514	**0.611**	0.522	0.297	0.297	**0.329**	0.315
2	0.176	0.178	0.160	**0.181**	0.049	**0.060**	0.047	0.050
3	0.112	0.115	**0.117**	0.113	0.028	**0.039**	0.030	0.029
4	0.121	0.109	0.112	**0.129**	0.032	**0.041**	0.031	0.033
5	0.137	**0.144**	0.134	0.137	0.038	**0.046**	0.037	0.038
6+	**0.150**	0.068	0.109	0.143	**0.035**	0.029	0.029	0.033

Ablation 1: Top-k budget. Increasing k from 1 to 50 in node-centric retrieval improves Hit@k (trivially) but MRR increases only from 0.30 to 0.36 at 1-hop. The answer's rank does not improve; wider retrieval simply casts a broader net.

Ablation 2: k-hop expansion depth. For subgraph-centric retrieval, $k = 1$ achieves the best 1-hop performance (Hit@10 = 0.64). Expanding to $k = 2$ hops *hurts* 1-hop performance (0.48) while providing only a modest gain at 2-hop (0.30 vs. 0.24). This is explained by the graph's high mean degree: a 2-hop expansion from 3 seed nodes potentially touches ∼3,600 nodes, flooding the subgraph with irrelevant context. Relevance-based pruning partially mitigates this but cannot overcome the signal-to-noise collapse.

Ablation 3: Path length. Shorter paths ($\ell = 2$) yield higher MRR at 2-hop (0.107) compared to longer paths ($\ell = 5$, MRR = 0.089). In a dense graph where many paths exist between any two nodes, shorter, more direct connections carry more information.

Ablation 4: Number of seed nodes. This ablation reveals the clearest tradeoff. With a single seed, subgraph-centric retrieval achieves its peak 1-hop performance (Hit@10 = 0.77, MRR = 0.45)—the highest in any experiment—but *completely zeros out* at 2+ hops. Increasing to 10 seeds helps multi-hop (Hit@10 = 0.26 at 2-hop) but reduces 1-hop to 0.46. No single seed count works across complexity levels.

Ablation 5: Hybrid text weight. The optimal mixture weight α shifts with query complexity: $\alpha = 0.2$ (mostly structural) is best for 1-hop (Hit@10 = 0.57), while $\alpha = 0.5$–0.6 (balanced) is best for multi-hop. Pure structural ($\alpha = 0.0$) performs worst overall, confirming that textual similarity provides essential signal even when graph structure is available.

4.4 Can the Cliff Be Avoided?

Motivated by the focus-vs-coverage tradeoff, we implement two additional strategies designed to overcome the cliff: embedding-guided beam search and query-adaptive routing.

Beam search. We sweep beam width $b \in \{3, 5, 10, 20\}$ and maximum hops $h \in \{1, 2, 3, 4\}$ on 200 samples per hop bin. Table 4 shows that beam search performs nearly identically to baselines across all configurations, with no significant improvement at any hop count.

Table 4: Beam search results (Hit@5 / MRR) compared to baselines. Beam width and hop depth make negligible difference.

	1-hop		2-hop	
Config	Hit@5	MRR	Hit@5	MRR
Node-centric (baseline)	0.400	0.354	0.195	0.085
Subgraph (k=1, baseline)	**0.480**	**0.390**	0.175	0.077
Beam (b=3, h=3)	0.435	0.365	0.190	0.085
Beam (b=10, h=3)	0.410	0.363	0.190	0.086
Beam (b=20, h=3)	0.410	0.359	**0.200**	**0.087**
Beam (b=10, h=4)	0.410	0.360	0.195	0.086

The failure of beam search reveals the *bridge entity problem*: intermediate nodes on the correct reasoning path (e.g., a gene connecting a disease to a drug) are not textually similar to the query. Embedding-guided pruning at each hop discards exactly the bridge entities needed to reach multi-hop answers.

Adaptive routing. We sweep the skewness threshold $\tau \in \{0.5, 1.0, 1.5, 2.0\}$, routing queries with skewness $> \tau$ to the focused configuration and the rest to the broad configuration. Table 5 shows the results.

Table 5: Adaptive router results. The router interpolates between focused and broad configs but score skewness does not correlate well with hop count.

	1-hop		2-hop		
Threshold τ	Hit@10	MRR	Hit@10	MRR	% Focused (2-hop)
0.5 (mostly focused)	**0.765**	**0.449**	0.005	0.002	97%
1.0	0.720	0.432	0.075	0.020	71%
1.5	0.660	0.408	0.180	0.052	32%
2.0 (mostly broad)	0.565	0.383	0.230	0.066	13%

The router reveals two problems. First, *skewness does not correlate well with hop count*: at $\tau = 1.0$, 71% of 2-hop queries are misrouted to the focused mode. The score distribution looks similar regardless of true query complexity because embedding similarity reflects topical relevance, not structural distance. Second, the router can only interpolate between the focused and broad extremes—it creates no new retrieval capability, merely blending existing ones.

Implications. The failure of both approaches confirms that the hop-count cliff is not an artifact of naive single-shot retrieval. Iterative expansion with embedding-guided pruning (beam search) and query-adaptive strategy selection (routing) both fail because they rely on the same underlying signal—embedding similarity—which does not capture relational reasoning.

4.5 Cross-Domain Replication: The Role of Graph Density

To test whether the cliff is specific to PrimeKG or a general phenomenon, we replicate on STaRK-Amazon, a much sparser graph (mean degree 9.1 vs. 62.6). Figure 2 compares performance degradation across the two datasets.

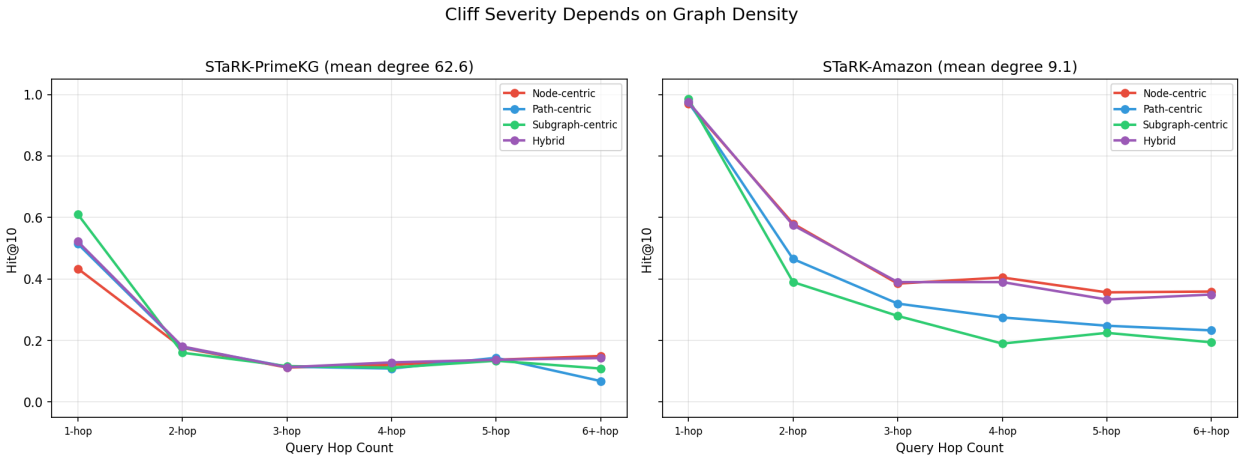

Figure 2: Hit@10 vs. hop count on PrimeKG (dense, left) and Amazon (sparse, right). The cliff is severe on dense PrimeKG but gradual on sparse Amazon.

Table 6 shows the Amazon results. Three striking differences emerge:

Table 6: Hit@10 by hop count and strategy on STaRK-Amazon ($n = 200$/bin). Bold indicates best per row.

Hop	Node	Path	Subgraph	Hybrid
1	0.970	0.975	**0.985**	0.975
2	**0.580**	0.465	0.390	0.575
3	0.385	0.320	0.280	**0.390**
4	**0.405**	0.275	0.190	0.390
5	**0.357**	0.248	0.225	0.333
6+	**0.359**	0.233	0.194	0.350

1. **The cliff is softer.** Hit@10 drops from 0.98 to 0.58 at 2-hop (∼40% drop) compared to PrimeKG's 70% drop. Performance degrades gradually rather than catastrophically (Figure 3).

2. **Node-centric beats structural methods at multi-hop.** On dense PrimeKG, subgraph-centric excels at 1-hop. On sparse Amazon, *node-centric outperforms all structural methods* at 2+ hops. Structural expansion on a sparse graph adds too few neighbors (∼9 per hop) to provide meaningful coverage, while potentially introducing noise.

3. **Subgraph-centric is worst at multi-hop**—the opposite of its PrimeKG behavior. This confirms that the value of structural retrieval depends critically on graph density.

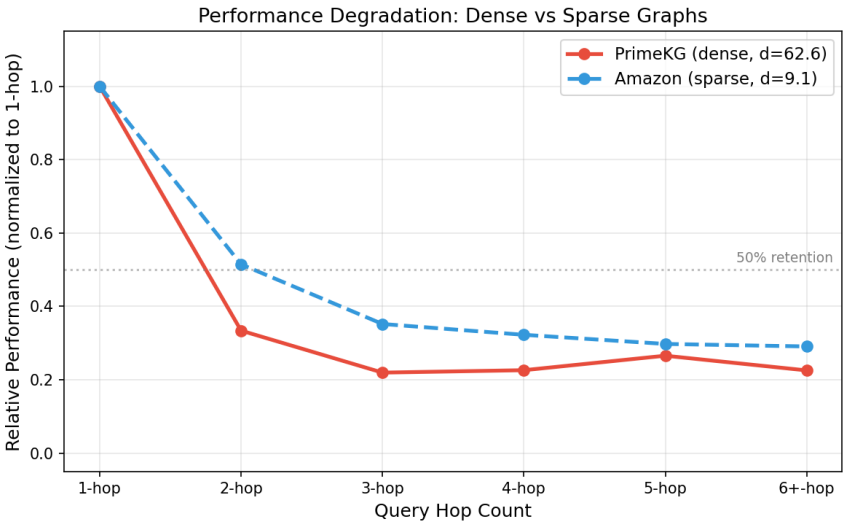

Figure 3: Performance retention relative to 1-hop (averaged across strategies). Dense PrimeKG loses 70% of performance at 2-hop; sparse Amazon retains 60%.

5 Discussion

5.1 Why the Cliff?

The hop-count cliff is a consequence of graph density. On STaRK-PrimeKG, the mean degree is 62.6, meaning a single hop from a query entity reaches ∼60 nodes. At 1-hop, the answer is likely within this immediate neighborhood, and all structural methods can find it. At 2-hop, the answer is somewhere among ∼3,600 second-degree neighbors, and embedding similarity alone cannot reliably distinguish the correct answer from thousands of structurally similar alternatives.

The Amazon replication confirms this explanation: with mean degree 9.1, a 2-hop expansion touches only ∼80 nodes, and the cliff is correspondingly softer. The relationship between density and cliff severity is consistent with a combinatorial argument: the number of candidate nodes at hop h scales as $\bar{d}^h$, and embedding-based ranking fails when this number exceeds the effective discriminative capacity of the embedding space.

This also explains why structural methods hurt on sparse Amazon: when the neighborhood is small, expanding it adds little information but can introduce noise. The benefit of structure requires a "Goldilocks zone" of density where expansion reaches the answer but doesn't overwhelm the ranker.

5.2 The Focus-vs-Coverage Tradeoff

Our ablations reveal that every hyperparameter controlling retrieval breadth (number of seeds, expansion depth, path length, text weight) exhibits the same pattern: settings that maximize 1-hop performance minimize multi-hop performance and vice versa. This is not a limitation of our specific implementations but a structural property of retrieval on dense graphs: *the information needed to reach multi-hop answers is diffused across many graph paths, and any strategy that focuses enough to avoid noise at 1-hop will miss the signal at higher hops.*

5.3 Connections to Prior Work

Our results provide empirical grounding for observations in the literature:

- Yue et al. [2025]'s "uneven performance across query logics" is not merely uneven—it is a cliff, with fundamentally different regimes above and below 2 hops.
- Yu et al. [2025]'s finding that GraphRAG "doesn't always retrieve what you need" is explained by the density-driven noise that overwhelms structural signal at higher hop counts.
- The success of Yue et al. [2025]'s planning stage can be understood as an implicit mechanism for adapting retrieval breadth to estimated query complexity.

5.4 Implications

Our results suggest that effective GraphRAG on dense knowledge graphs cannot be solved by better embedding-based strategies alone—whether fixed, iterative, or adaptive. The cliff persists because embedding similarity does not encode relational reasoning. Two directions appear promising:

1. **Relation-type-aware traversal**: On typed KGs like PrimeKG, filtering edges by predicted relation type before expansion could reduce the effective degree from $\sim$60 to $\sim$10 per hop. Methods like Think-on-Graph [Sun et al., 2024] and MoR's planning stage [Yue et al., 2025] implement variants of this idea.
2. **Iterative retrieval-generation**: Interleaving retrieval with LLM reasoning [Trivedi et al., 2023] decomposes multi-hop problems into sequential single-hop retrievals, each guided by the current reasoning state. This avoids the neighborhood explosion entirely by never expanding more than one hop at a time.

Both directions require LLM calls during retrieval, trading computational cost for reasoning capability. Whether this trade-off is worthwhile depends on the density of the target graph and the proportion of multi-hop queries.

5.5 Limitations

Our retrievers are relatively simple implementations that serve as controlled baselines rather than state-of-the-art systems. The papers we build on use more sophisticated approaches (learned planners, GNN encoders, LLM-guided traversal). Our contribution is characterizing the hop-count cliff and the density dependence, not claiming that our implementations are optimal.

While we evaluate on two domains (biomedical and e-commerce) spanning a wide range of graph densities, our density comparison is between two points. A more thorough investigation would sweep density continuously, e.g., by subsampling edges from a dense graph.

Our hop-count stratification relies on identifying the query entity via embedding similarity, which introduces noise. Some queries may be assigned to incorrect hop-count bins, potentially smoothing the observed cliff. The true cliff may be even sharper than we report.

6 Conclusion

We set out to find a phase transition in GraphRAG retrieval performance and found something more nuanced: a *density-dependent cliff*. On dense knowledge graphs (PrimeKG, $\bar{d} = 62.6$), all

six retrieval strategies collapse at 2-hop queries with a ∼70% performance drop. On sparse graphs (Amazon, $\bar{d} = 9.1$), the degradation is gradual (∼40%), and structural methods can even *hurt*—the opposite of dense-graph behavior.

Through systematic ablation studies, we show that the cliff arises from a fundamental focus-versus-coverage tradeoff driven by the combinatorial explosion of neighborhoods on dense graphs. Through beam search and adaptive routing experiments, we show the cliff is robust to principled attempts to overcome it within the embedding-based paradigm. The bridge entity problem—intermediate reasoning nodes are not textually similar to the query—renders embedding-guided traversal ineffective for multi-hop reasoning regardless of architecture.

The cross-domain comparison reveals graph density as the key moderating variable for GraphRAG system design. On sparse graphs, simple text-based retrieval suffices even for multi-hop queries. On dense graphs, overcoming the cliff requires moving beyond embeddings to *relation-type-aware* methods that leverage edge semantics, or to iterative retrieval-generation loops that decompose multi-hop reasoning into manageable single-hop steps.

References

Chandak, P., Huang, K., and Zitnik, M. Building a knowledge graph to enable precision medicine. *Scientific Data*, 10(67), 2023.

Hu, Y., Lei, Z., Zhang, Z., Pan, B., Ling, C., and Zhao, L. GRAG: Graph Retrieval-Augmented Generation. *arXiv preprint arXiv:2405.16506*, 2024.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 2020.

Sun, J., Xu, C., Tang, L., Wang, S., Lin, C., Gong, Y., Shum, H., and Guo, J. Think-on-Graph: Deep and responsible reasoning of large language model on knowledge graph. *ICLR*, 2024.

Trivedi, H., Balasubramanian, N., Khot, T., and Sabharwal, A. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. *ACL*, 2023.

Wang, H., et al. SkewRoute: Training-free LLM routing for KG-RAG via score skewness of retrieved context. *Findings of EMNLP*, 2025.

Reimers, N. and Gurevych, I. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *EMNLP*, 2019.

Wu, S., et al. STaRK: Benchmarking LLM retrieval on textual and relational knowledge bases. *NeurIPS Datasets and Benchmarks*, 2024.

Yu, J., Liu, Y., Gu, J., Torr, P., and Zhou, D. Can knowledge-graph-based retrieval augmented generation really retrieve what you need? *NeurIPS*, 2025.

Yue, Z., et al. Mixture of structural-and-textual retrieval over text-rich graph knowledge bases. *Findings of ACL*, 2025.